Chair of
Information Theory
and Data Analytics

RWTH AACHEN
UNIVERSITY

RESOURCE-EFFICIENT
INFERENCE IN
LARGE GENERATIVE MODELS

Patrick Tazo Kuete

A seminar submitted to the
Faculty of Electrical Engineering and Information Technology,
RWTH Aachen University,

supervised by Hazem el Hadj Khalifa, M.Sc.

Chair of Information Theory and Data Analytics
RWTH Aachen University

Abstract

Large generative models, in particular decoder-only Transformer architectures, have demonstrated unprecedented capabilities in natural language processing, code synthesis, and multimodal reasoning. However, the inference process is computationally intensive: repeated autoregressive forward passes impose severe demands on memory, compute, memory bandwidth, and energy. These characteristics make deployment on resource-constrained edge devices such as smartphones, embedded systems, and Internet of Things (IoT) hardware a significant open challenge.

This report provides a structured survey of resource-efficient inference in large generative models. Starting from the fundamental Transformer building blocks, it traces the complete inference pipeline and systematically identifies the principal resource bottlenecks at each stage, distinguishing between the compute-bound prefill phase and the memory-bandwidth-bound decode phase. Building on this analysis, the paper surveys the leading efficiency techniques across four layers: model compression (quantisation, pruning, and knowledge distillation), inference-level algorithmic improvements (optimised attention mechanisms and Key-Value (KV) cache management), decoding strategies (speculative decoding and batching), and system-level runtimes (vLLM, llama.cpp). For each technique, the discussion focuses on the specific bottleneck it addresses and the practical trade-offs that arise when targeting edge hardware.

Contents

List of Figures

List of Abbreviations

AI Artificial Intelligence 1

BLAS Basic Linear Algebra Subprograms 10

CPU Central Processing Unit 10

DRAM Dynamic Random Access Memory 16, 17

FFN Feed-Forward Network 10

FLOP Floating Point Operation 10

FP16 16-bit Floating Point 17

GPU Graphics Processing Unit 10, 12, 14

GQA Grouped-Query Attention 16

INT8 8-bit Integer 12

IoT Internet of Things iii

KD Knowledge Distillation 10

KV Key-Value iii, 5, 6, 12, 15–17

LLM Large Language Model 1, 7, 14, 17

MLC Machine Learning Compilation 14

TPOT Time per Output Token 6

TTFT Time to First Token 6

1 Introduction

The past decade has witnessed a remarkable transformation in Artificial Intelligence (AI) research, driven by the emergence and rapid scaling of large generative models. Systems such as GPT-3 [2], LLaMA [10], and their successors have demonstrated near-human performance on a broad spectrum of tasks—from natural language understanding and code generation to mathematical reasoning and multimodal dialogue. This progress rests fundamentally on the Transformer architecture [11], which, when trained on trillions of tokens and scaled to hundreds of billions of parameters, exhibits surprising emergent capabilities.

Yet capability has come at a steep cost. Generating even a single response with a state-of-the-art Large Language Model (LLM) requires billions of floating-point operations, gigabytes of memory, and significant electrical power. In data-centre settings this cost is absorbed through specialised hardware and economies of scale, but it nonetheless drives substantial economic and environmental expenditure. On resource-constrained platforms—smartphones, embedded controllers, autonomous vehicles—such demands can make deployment entirely infeasible without dedicated optimisation. The field of resource-efficient LLM inference has therefore emerged as one of the most active and practically consequential research areas in machine learning [1].

This paper provides a structured survey of the state of the art in resource-efficient inference for large generative models. It begins by grounding the analysis in a concrete understanding of what makes the present day Large Generative Models so powerful—Transformers—and goes on to briefly explain the process of inference, highlighting where applicable, which resources are the limiting factor. In addition to this, a short list of terms deemed important will be defined, to give a clear foundation chapter 2. As the main course of this Paper, a comprehensive survey of efficiency techniques follows in chapter 3. This survey is structured around the three main axes of efficiency: model compression, algorithmic optimisation, and hardware acceleration. Each technique is described in terms of its underlying principles, practical implementation considerations, and empirical performance trade-offs. Finally, the paper concludes with a discussion of open challenges and future directions in the field chapter 4.

2 Preliminaries and Taxonomy

2.1 Transformer Architecture

The Transformer model has brought considerable advancements to the field of Natural Language Processing (NLP). Unlike sequential traditional models such as RNNs (Recurrent Neural Networks), the Transformer architecture uses an innovative attention mechanism to keep track of long-range dependencies between tokens derived from the input sequence. This is enabled by a clever mechanism, which enriches the token representations (Embeddings) by taking into consideration the meaning of the other surrounding tokens: Attention [11]

Prior to enriching the input tokens with contextual meaning, the latter is first mapped to a vector in a d_k dimensional space where the meaning of the word is encoded in a first instance. This is known as Embedding, and the resulting vectors are often referred to as input embeddings. Following this, knowledge about of the position of each token is also needed and this information is represented using positional encodings, usually generated by sinusodial funtions. These positional encodings are then added to the input embeddings, as Transformers do not inherently have knowledge of the position of each token.

The Attention mechanism for a given set of input tokens starts by deriving three vectors for each token, namely: the Query, Key and Value vectors by making a linear projection with learned projection matrices. A dot product between all the query and key vectors of each input embedding is then computed to find out which tokens are compatible (in the formular below, this is expressed as a matrix multiplication of the Query and Key matrices). In order to prevent the dot products to grow too large and push the softmax function into areas where the gradients are extremely small, the dot products are normalised by $\sqrt{d_k}$. The values obtained after the softmax are then used as weights which are multiplied with each value vector and added

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where, Q, K, and V represent the query, key, and value matrices, which are a compact representation for the different query, key and value vectors of all input tokens, and d_k is the dimension of the key vectors.

as previously mentioned, builds on this self-attention mechanism, by carrying out the same operation in parallel with different learned weights and concatenating the output of each individual head, In order to capture other possible relationships between the input tokens which would not necessarily be captured by a single attention head, the Multi-Head Self-Attention is introduced. This extends the Self-Attention mechanism by computing different Attention heads in parallel for different sets of learned projection matrices of the Query, Key and Value and then concatenating the results. This can be represented as follows:

$$MultiHead(Q, K, V) = Concat(head_1, head_2, ..., head_h)W^O$$

where $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$ stands for the i-th attention head output, and W^O is the output projection matrix. [1]

As can be seen from the attention formular above, the matrix multiplication between Q and K^T, each of dimension $(n \times d_k)$ and $(d_x \times n)$ respectively, brings about a time complexity of $O(n^2)$, where n denotes the input sequence lenght. The subsequent multiplication with the value matrix accounts for a complexity of $O(d)$. Putting these two together, we see that the time complexity for the Attention mechanism amounts to $O(n^2 d)$. The time complexity of the Attention mechanism therefore grows quadratically with the input sequence, which could bring about computational difficulties for verly long input sequences. Other than time complexity, it is worth taking a look at memory complexity as well. This amounts to $O(n^2)$ as result of the $(n \times n)$ attention matrix (QT^k) needing to be stored for further computation.

2.2 Inference in LLMs

The inference pipeline in Transformer decoder based LLMs is made up of mainly two stages: the Prefill Stage and Decode Stage

The prefill stage consists of creating Key-Value (KV) pairs from the input tokens, which will later be cached (KV-cache). These are used to determine the contextual importance of one token when compared with another. At this point it is worth highlighting that the size of the KV-cache grows with input length, which could easily strain memory. [14]

The Decode Stage consists of computing new Key and Value pairs based on the output tokens generated and appending these to the alreading existing KV-cache. [13]

A brief overview of the how inference works in LLMs, can be seen in Fig. 2.1

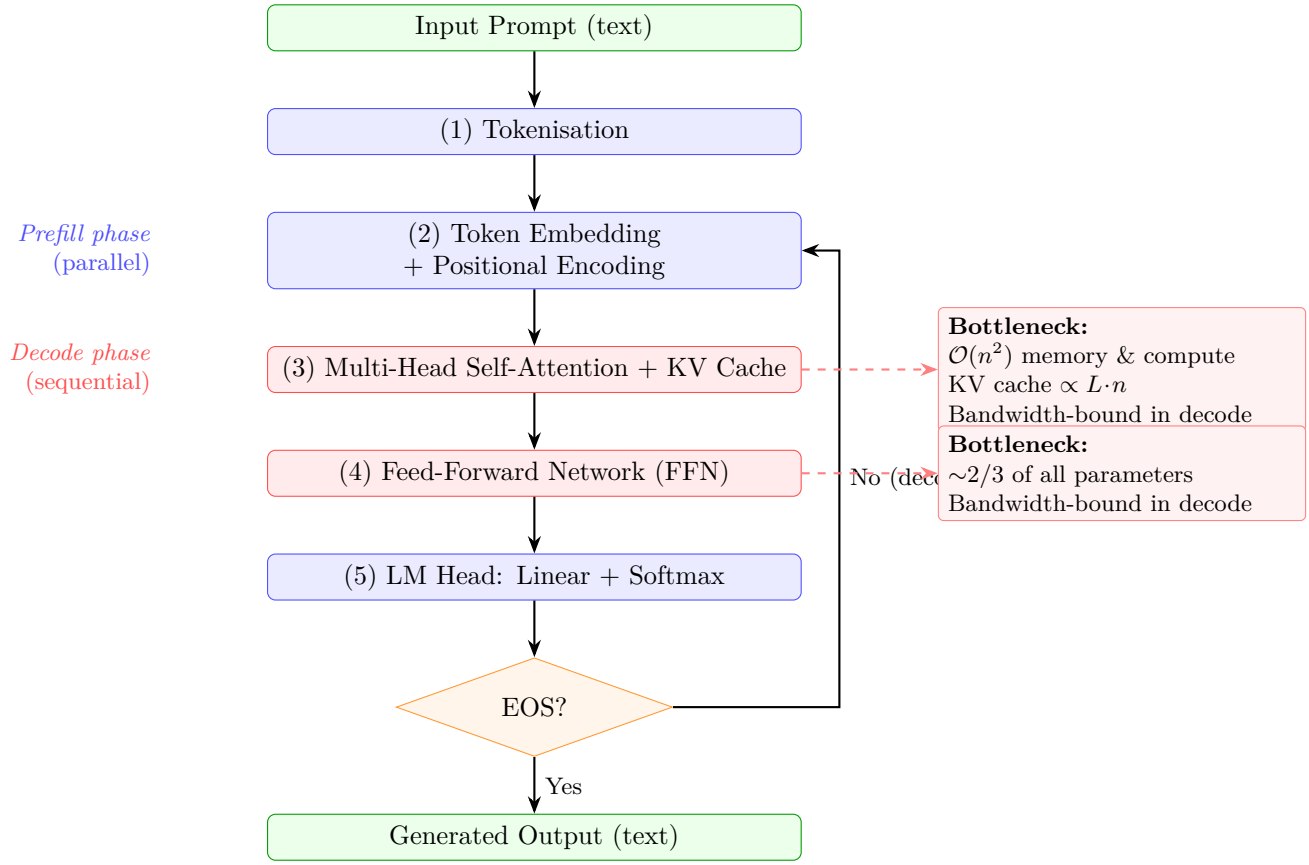

Figure 2.1: Schematic of the Transformer inference pipeline. Red-shaded steps (3, 4) represent the primary resource bottlenecks. The decode loop repeats steps 3–5 until an end-of-sequence token is produced.

2.3 Taxonomy and Key Metrics

The following terms are consistently throughout this report.

Computation: Computation refers to the amount of arithmetic processing required to perform inference. It is typically measured by the number of floating-point operations (FLOPs) or integer operations needed to generate an output.

Memory: Memory refers to the amount of storage required to hold the data needed during inference, including model parameters, intermediate activations, and the key-value (KV) cache, measured in Bytes.

Energy: Total amount of electircal energy consumed during the inference process measured in Joules.

Bandwidth: maximum amount of data that can be transferred between memory and the processor per unit time. In our case, this could for example be how fast the parameters of the LLM are loaded into the GPU.

KV cache: The stored key and value tensors for all past tokens, maintained to avoid recomputation at each decode step.

Quantization: Representing weights and/or activations at reduced numerical precision to reduce memory footprint and, on hardware with native low-precision arithmetic, increase throughput.

Pruning: Removing parameters from a trained model. *Unstructured* pruning zeros individual weights; *structured* pruning removes entire rows, columns, heads, or layers to yield a dense, smaller model directly executable on standard hardware.

Knowledge Distillation: Training a compact *student* model to match the output distribution of a larger *teacher*, transferring capability at a fraction of the inference cost.

Throughput: Tokens (or requests) completed per second across all concurrent users measured in tokens per second.

Latency: Elapsed time from request arrival to final-token delivery. Time to First Token (TTFT) (time to first token) governs perceived responsiveness; Time per Output Token (TPOT) (time per output token) governs generation speed.

Edge device: Any computing platform with constrained memory, power, and compute operating outside a data centre: smartphones or embedded controllers.

3 Towards Efficient Inference: Techniques and Solutions

Building upon the previous chapter, this chapter systematically surveys the techniques that have been developed to reduce the resource demands of LLM inference. For each technique the discussion covers its core mechanism, the bottleneck it addresses, its accuracy–efficiency trade-off, and its practical implications for edge deployment. figure 3.1 provides an overview of how the techniques presented in this chapter map to the bottlenecks identified they address.

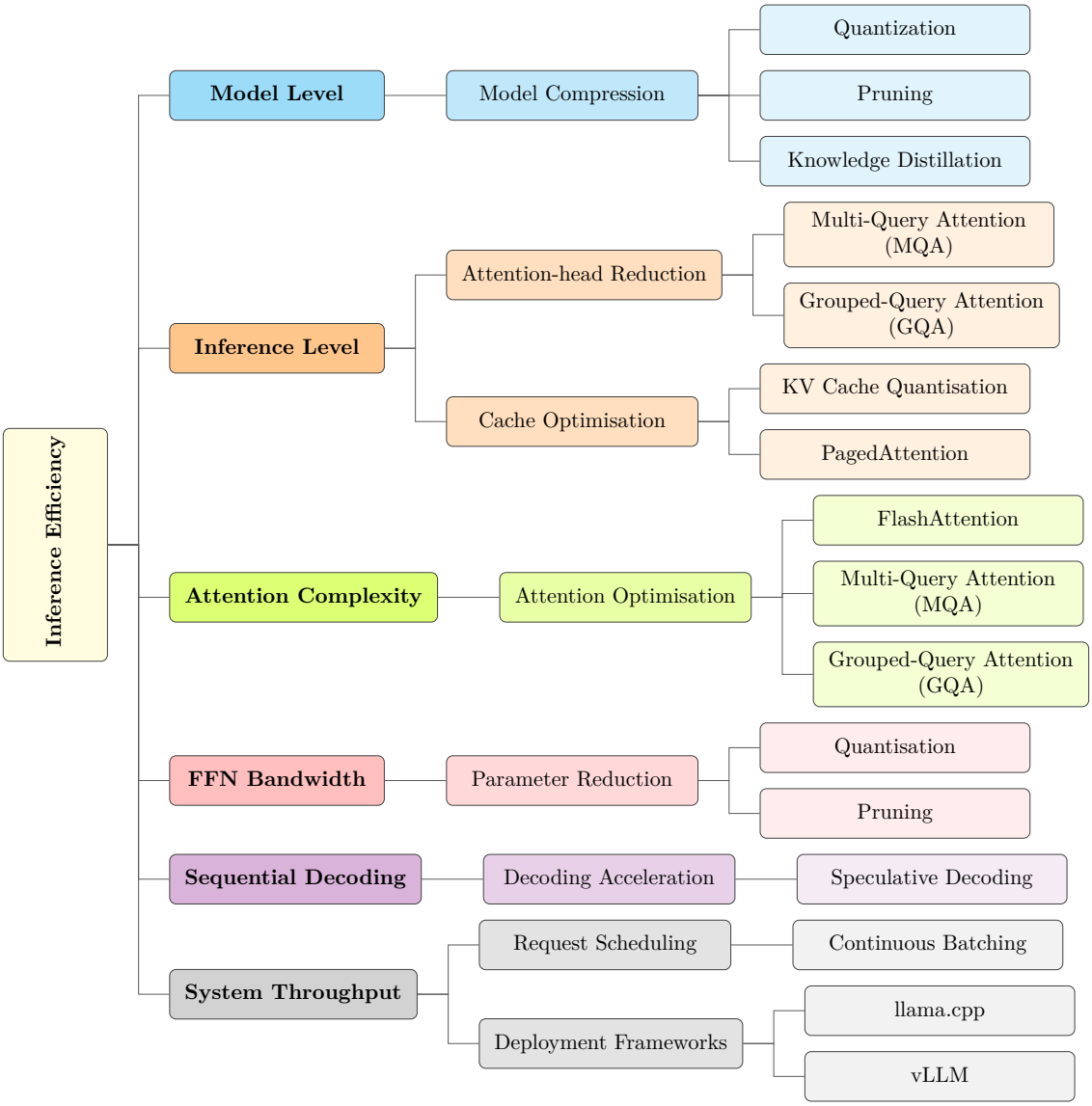

Figure 3.1: Hierarchical classification of major inference bottlenecks and the techniques used to address them. Techniques appearing in multiple branches target more than one bottleneck.

3.1 Model Compression

Model compression techniques reduce the size of the model itself—shrinking weight memory and, consequently, the bandwidth required to load those weights during each decode step. They are pre-deployment operations and impose a one-time cost in compute or data for calibration.

3.1.1 Quantisation

Quantization is a model-compression strategy that lowers the numerical precision of model parameters and/or intermediate values. Instead of storing and computing with high-precision formats such as FP32 or FP16, quantized models use lower-bit formats such as INT8, INT4, or even lower representations. This is especially useful for edge deployment, where memory capacity, bandwidth, energy consumption, and computational resources are limited. However, quantizing large language models is more difficult than quantizing many conventional neural networks because Transformer architectures contain attention layers, high-dimensional hidden states, and activation distributions with large dynamic ranges. These properties make LLMs sensitive to numerical approximation, so aggressive quantization can reduce model quality if it is not carefully controlled. [12]

Existing LLM quantization methods can be broadly divided into two categories: weight-only quantization and weight-activation co-quantization. Weight-only quantization reduces the precision of the model's stored parameters while usually keeping activations in higher precision. This mainly lowers memory usage and can improve inference efficiency. Examples include LLM.int8(), which enables 8-bit matrix multiplication for Transformer layers, GPTQ, which performs post-training weight quantization using approximate second-order information, and AWQ, which identifies important weight channels using activation statistics and protects them from excessive quantization error. These methods show that substantial compression is possible, but handling outlier or highly sensitive weights remains an important challenge. [12]

Weight-activation co-quantization goes further by reducing the precision of both weights and intermediate activations. This can provide additional memory and computational savings, but it is harder because activations in LLMs often contain large outliers and irregular value ranges. Methods such as ZeroQuant and SmoothQuant address this by using more refined scaling or grouping strategies. In particular, SmoothQuant shifts part of the quantization difficulty from activations to weights through mathematically equivalent scaling transformations, enabling efficient 8-bit weight-and-activation quantization with limited accuracy loss. Overall, quantization is one of the most important techniques for making LLM inference feasible on edge devices, but its effectiveness depends on balancing compression, hardware efficiency, and preservation of model accuracy. [12]

3.1.2 Pruning

Pruning removes parameters from the model that contribute least to its predictions, reducing both weight memory and Floating Point Operation (FLOP) count. The two principal strategies—*unstructured* and *structured* pruning—differ in *what* they remove and, consequently, in the hardware speedup they deliver. Figure figure 3.2 illustrates both approaches on a concrete 4×4 weight matrix.

Unstructured pruning zeros individual weights according to a saliency criterion; the most widely used is *magnitude pruning*: weight w_{ij} is pruned if $|w_{ij}| < \theta$ for a chosen threshold θ. As illustrated in figure 3.2(b), applying $\theta = 0.25$ to $\mathbf{W}$ zeros seven of the sixteen entries (44% sparsity) while retaining the nine larger-magnitude weights intact. The critical limitation is that the resulting sparse matrix is *structurally identical* to the original: non-zero values follow no regular pattern, so the data must be encoded in a sparse format (e.g., Compressed Sparse Row), and standard dense Basic Linear Algebra Subprograms (BLAS) routines cannot skip the zeroed positions. Practical speedups therefore require hardware support for sparse arithmetic—available on NVIDIA Ampere Graphics Processing Units (GPUs) via 2:4 structured sparsity but largely absent from edge Central Processing Units (CPUs). Despite this constraint, high sparsity levels (80–90%) can be achieved with minimal quality loss for moderately sized models [1], making unstructured pruning attractive when the inference platform supports sparse matrix operations.

Structured pruning removes entire, regular sub-structures—rows or columns of weight matrices, attention heads, Feed-Forward Network (FFN) intermediate dimensions, or whole Transformer layers—yielding a strictly smaller *dense* matrix that requires no sparse encoding and executes efficiently on any standard hardware. As shown in figure 3.2(c), the row with the smallest ℓ_2 norm ($\mathbf{r}_1$, norm ≈ 0.78) is eliminated entirely: the 4×4 matrix shrinks to a 3×4 matrix, delivering a genuine 25% reduction in both parameter count and multiply–accumulate operations with no specialised kernel required.

3.1.3 Knowledge Distillation

Knowledge distillation (KD) [6] trains a smaller *student* model to mimic the behaviour of a Knowledge Distillation (KD) is a model compression technique in which a smaller student model is trained to imitate the behavior of a larger teacher model. In the context of large language models, KD is used to transfer knowledge and capabilities from powerful LLMs into smaller language models, reducing memory and computational requirements while preserving as much performance as possible. KD methods for LLMs are commonly divided into white-box and black-box approaches. [14]

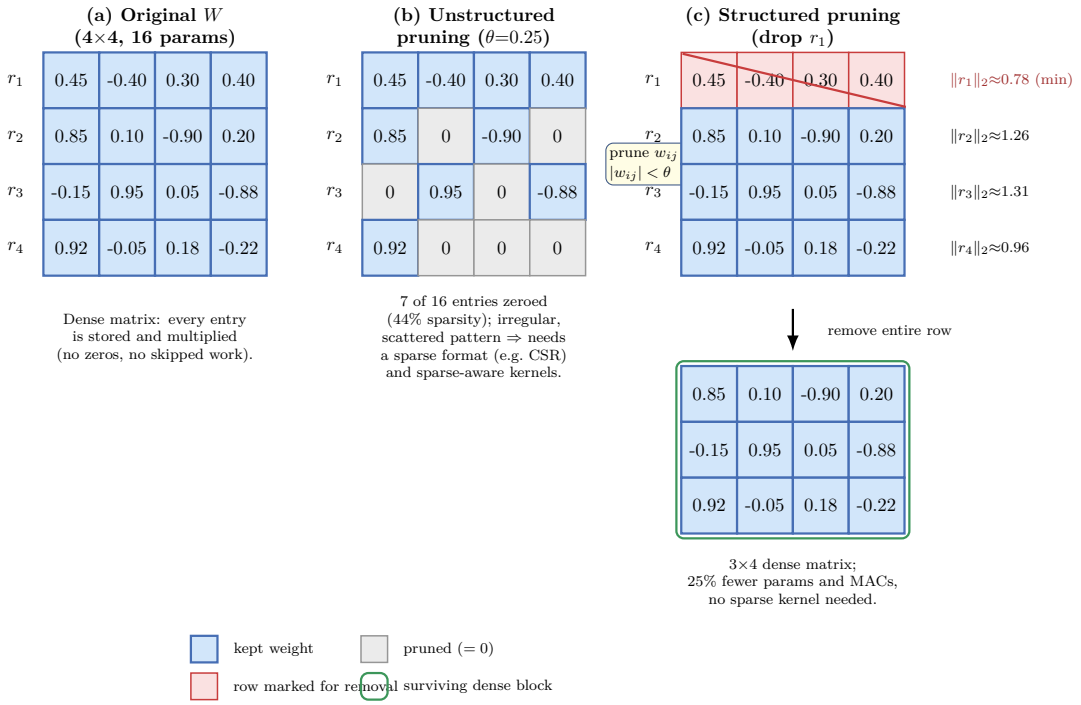

Figure 3.2: Comparison of unstructured and structured pruning strategies on a 4×4 weight matrix. (a) The original dense matrix stores and multiplies all 16 parameters. (b) Unstructured magnitude pruning zeros individual weights below threshold θ=0.25, achieving 44% sparsity but requiring sparse matrix formats and specialised kernels. (c) Structured pruning removes the entire row with smallest ℓ_2 norm, yielding a 3×4 dense matrix that delivers genuine 25% savings in both parameters and operations without specialised hardware support.

White-box KD assumes access to the internal components of the teacher model, such as its parameters, hidden representations, intermediate features, or output logits. This allows the student model to learn not only from the teacher's final predictions, but also from its internal computation process. Existing methods use different strategies, such as aligning teacher and student logits, matching layer-wise features, training with student-generated data, reducing model width and depth, or using specialized student architectures to reduce the capacity gap between teacher and student models. [14]

Black-box KD is applied when the internal structure and parameters of the teacher model are unavailable, as is often the case with API-based or proprietary LLMs. In this setting, the student model is trained only on the teacher's generated outputs, such as answers, rationales, demonstrations, or instruction-response pairs. Black-box KD is therefore mainly used to transfer high-level LLM abilities, including in-context learning, chain-of-thought reasoning, and instruction following, by encouraging the smaller model to imitate the observable behavior of the larger model. [14]

3.2 Inference-Level Optimisations

These techniques modify the attention mechanism or the KV cache management strategy without altering the model weights. They can be applied at inference time, often transparently to the model.

3.2.1 KV Cache Optimisation

Paged Attention [7], implemented in the vLLM framework, adapts the virtual memory paging concept to the KV cache. Each request's KV vectors are stored in fixed-size *pages* that need not be physically contiguous. A page table maps logical to physical page addresses at inference time. This eliminates nearly all internal and external fragmentation, increasing the effective number of concurrent requests that can be served from the same GPU memory by 2–4×.

KV cache quantisation. The KV cache can itself be stored at reduced precision. Quantising the cached key/value tensors to 8-bit Integer (INT8) halves cache memory with minimal perplexity impact. More aggressive quantisation to INT4 is possible with careful calibration, providing a further 2× reduction.

KV cache eviction. For very long sequences, it is possible to *evict* (discard) cache entries for tokens that are unlikely to be attended to by future queries, based on heuristics such

as attention score recency or magnitude. StreamingLLM and similar methods maintain a fixed-size cache window, enabling effectively unbounded context length at the cost of some context fidelity.

3.3 Decoding Strategies

3.3.1 Speculative Decoding

Standard autoregressive decoding generates one token per forward pass, enforcing a strict latency lower bound of (number of generated tokens) $\times$ (time per forward pass). This is wasteful when the full model is large but most token generations are 'easy' (highly predictable).

Speculative decoding [8] exploits this insight with a two-model approach:

1. A small, fast *draft model* (e.g., 1–3B parameters) generates K candidate tokens autoregressively in K forward passes.

2. The large *target model* processes all K draft tokens in a single forward pass (which is parallelisable, like prefill) and accepts or rejects each draft token using a rejection sampling criterion that guarantees the accepted sequence has the same distribution as the target model's output.

When the draft model's predictions match the target with probability α (the *acceptance rate*), the expected number of tokens produced per target-model forward pass is $\frac{1-\alpha^{K+1}}{1-\alpha}$. For $\alpha \approx 0.8$ and $K = 5$, this yields approximately 3.5 tokens per call to the target model—a 3.5$\times$ latency improvement with no loss in output quality [8].

For edge deployment, the 'large model' and 'draft model' may themselves be a quantised medium model and a tiny model respectively. The framework is flexible: any two models with matching tokenisers and vocabulary can be paired.

3.3.2 Continuous Batching

Traditional static batching waits until a fixed number of requests arrive and then processes them together. Requests of different lengths must be padded to a common length (wasting compute) and the entire batch must finish before new requests are admitted (increasing average latency).

Continuous (or iteration-level) batching inserts new requests into the running batch at the earliest available slot (e.g., whenever a request produces an EOS token and vacates its slot). This keeps hardware utilisation high and reduces median latency significantly. Combined with PagedAttention, continuous batching enables high-throughput serving that is relevant for cloud deployments and, increasingly, for shared edge inference servers in enterprise settings.

3.4 System-Level Runtimes and Edge Frameworks

Efficiency techniques are only realised in practice when they are implemented in well-engineered software runtimes that expose hardware capabilities efficiently. Several frameworks have emerged specifically for LLM inference, with differing strengths on server and edge hardware.

3.4.1 llama.cpp

`llama.cpp` [5] is a C/C++ inference runtime designed for deployment on commodity hardware, including ARM CPUs (Apple Silicon, Android), x86 CPUs, and consumer GPUs. Its key design decisions include:

- **GGUF quantisation format**: a family of mixed-precision weight quantisation schemes (2–8 bits per weight) with hand-tuned dequantisation kernels for NEON (ARM SIMD), AVX2, and CUDA.
- **Minimal dependencies**: the library has no deep learning framework dependency, making it straightforward to compile and deploy on embedded targets.
- **Metal / Vulkan backend**: hardware-accelerated compute on Apple Neural Engine and Mali/Adreno GPUs on mobile devices.

In practice, llama.cpp enables a quantised (Q4_K_M) 7B model to generate tokens at 20–40 tokens/second on a modern laptop CPU and at 60–80 tokens/second on an Apple M-series chip—sufficient for interactive use [**?**].

3.4.2 MLC-LLM

MLC-LLM [3] (Machine Learning Compilation (MLC) Large Language Model) takes a compilation-based approach. It uses the Apache TVM deep learning compiler to gener-

ate hardware-specific compute kernels from a high-level model description, targeting iOS (Metal), Android (OpenCL, Vulkan), WebGPU, CUDA, and ROCm from a single code base. Key advantages include:

- Auto-tuning of tiling and scheduling parameters for the target hardware.
- Support for both weight quantisation and KV cache quantisation.
- On-device inference on smartphones (e.g., LLaMA-2 7B at 3–8 tokens/second on a Pixel 7 Pro).

The compilation step adds several minutes of one-time overhead but produces deployment artefacts that run efficiently with no runtime compilation cost.

3.4.3 vLLM

vLLM [7] is optimised for *server-side* high-throughput serving. Its core contribution is PagedAttention (Section section 3.2.1), which dramatically reduces KV cache memory fragmentation. vLLM additionally implements continuous batching, FlashAttention, and quantised inference, providing a production-grade serving stack for data-centre deployment. While not directly targeted at edge hardware, vLLM establishes the baseline for cloud-side efficient inference from which edge-focused frameworks draw inspiration.

3.5 Discussion: Trade-offs and Edge-Specific Considerations

No single technique dominates across all deployment scenarios. The efficiency gains and quality losses described above interact in ways that must be understood holistically when targeting edge hardware.

3.5.1 The Efficiency–Accuracy Trade-off

Compression techniques invariably introduce some quality degradation. The magnitude of this degradation depends on:

- **Bit width**: quantisation from FP16 to INT8 is nearly lossless; from INT8 to INT4 introduces a noticeable but usually acceptable quality reduction; below INT4, degradation accelerates [4].

- **Task sensitivity**: arithmetic, logical reasoning, and mathematical tasks degrade faster under quantisation than open-ended text generation [1].

- **Model scale**: larger models exhibit greater robustness to compression. A 70B model quantised to INT4 often outperforms an uncompressed 7B model, making the question 'which compressed large model vs. which uncompressed small model' a central practical question for edge deployment.

3.5.2 The Fundamental Edge Constraint: Memory

Across all dimensions, memory remains the hardest constraint on edge devices. Compute and energy can often be traded against latency by reducing clock frequency; bandwidth can be partially mitigated by caching. But there is no recourse when a model simply does not fit in the available Dynamic Random Access Memory (DRAM): inference is not possible. This makes weight quantisation and architectural choices that reduce the KV cache (such as Grouped-Query Attention (GQA)) the most impactful interventions for edge deployment, not as performance optimisations but as enabling technologies [**?**].

4 Conclusion

This paper has traced the resource challenges of LLM inference from first principles through to practical deployment solutions, with a sustained focus on edge hardware as the most demanding deployment target. chapter 1 established that the four resource dimensions of LLM inference—compute, memory, memory bandwidth, and energy—are not equally constraining in all settings. On edge devices, memory capacity is the hard, non-negotiable limit: a model that does not fit in DRAM simply cannot run. Memory bandwidth is the dominant performance bottleneck during the decode phase, and energy determines the sustained throughput a device can deliver over time.

chapter 2 analysed the inference pipeline step by step and identified its two most resource-intensive components: the self-attention sub-layer, which incurs $O(n^2)$ compute during prefill and a growing KV cache during decode.

chapter 3 surveyed the state-of-the-art techniques for addressing these bottlenecks. Several key findings emerge from this analysis.

Quantisation is the single most impactful intervention for edge deployment. Reducing weight precision from 16-bit Floating Point (FP16) to INT4 achieves a 4× compression of model weights, making 7B-class models feasible on consumer hardware with 4–8 GB of memory. Methods such as GPTQ [4] and AWQ [9] demonstrate that this compression can be achieved with minimal perplexity degradation, particularly for models above 7B parameters.

Speculative decoding provides latency improvements without quality loss. For interactive applications—which are the primary use case for edge inference—reducing latency per generated token is more important than maximising throughput. Speculative decoding [8] directly targets this metric and is composable with other techniques.

Dedicated edge runtimes bridge the gap between algorithmic efficiency and hardware performance. llama.cpp [5] and MLC-LLM [3] demonstrate that practical edge inference is achievable with current 7B-class models when quantisation, hardware-specific SIMD kernels, and efficient memory management are integrated in a single, well-engineered stack.

Bibliography

[1] G. Bai, Z. Chai, C. Ling, S. Wang, J. Lu, C. Li, and L. Yao, "Beyond efficiency: A systematic survey of resource-efficient large language models," 2024. [Online]. Available: https://arxiv.org/abs/2401.00625

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[3] T. Chen *et al.*, "MLC-LLM: Universal LLM deployment engine with ML compilation," GitHub repository, 2023. [Online]. Available: https://github.com/mlc-ai/mlc-llm

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," 2023. [Online]. Available: https://arxiv.org/abs/2210.17323

[5] G. Gerganov and contributors, "llama.cpp: C/C++ port of Meta's LLaMA model," GitHub repository, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[6] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015. [Online]. Available: https://arxiv.org/abs/1503.02531

[7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[8] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning*, 2023. [Online]. Available: https://arxiv.org/abs/2211.17192

[9] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," in

Proceedings of Machine Learning and Systems, vol. 6, 2024. [Online]. Available: https://arxiv.org/abs/2306.00978

[10] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "LLaMA: Open and efficient foundation language models," 2023. [Online]. Available: https://arxiv.org/abs/2302.13971

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[12] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, "A review on edge large language models: Design, execution, and applications," *ACM Computing Surveys*, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3719664

[13] Y. Zhihang, S. Yuzhang, Z. Yang *et al.*, "Llm inference unveiled: Survey and roofline model insights," 2024. [Online]. Available: http://arxiv.org/abs/2402.16363

[14] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, and G. Dai, "A survey on efficient inference for large language models." 2024. [Online]. Available: https://arxiv.org/abs/2404.14294